

Министерство образования и науки РФ
Санкт-Петербургский политехнический университет Петра Великого
Институт компьютерных наук и кибербезопасности
Высшая школа программной инженерии

Отчет по курсовой работе
по дисциплине: «Объектно-ориентированное программирование»

Выполнила

студентка гр. в5130904/30321

Д.С. Шипоенко

Старший преподаватель

А.П. Маслаков

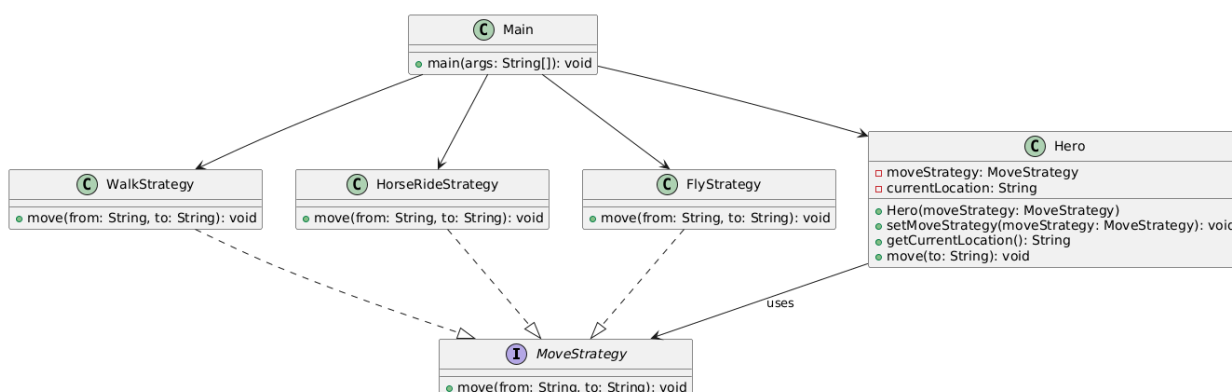
ОГЛАВЛЕНИЕ

ЗАДАНИЕ 1	3
Диаграмма классов.....	3
Перечень выполненных работ	3
ЗАДАНИЕ 2	4
Диаграмма классов.....	4
Перечень выполненных работ	4
ЗАДАНИЕ 3	5
Диаграмма классов.....	5
Перечень выполненных работ	5
ЗАДАНИЕ 4	7
Диаграмма классов.....	7
Перечень выполненных работ	7
КУРСОВАЯ РАБОТА.....	9
Диаграмма классов.....	9
Перечень выполненных работ	9

ЗАДАНИЕ 1

Паттерн Strategy

Диаграмма классов



Перечень выполненных работ

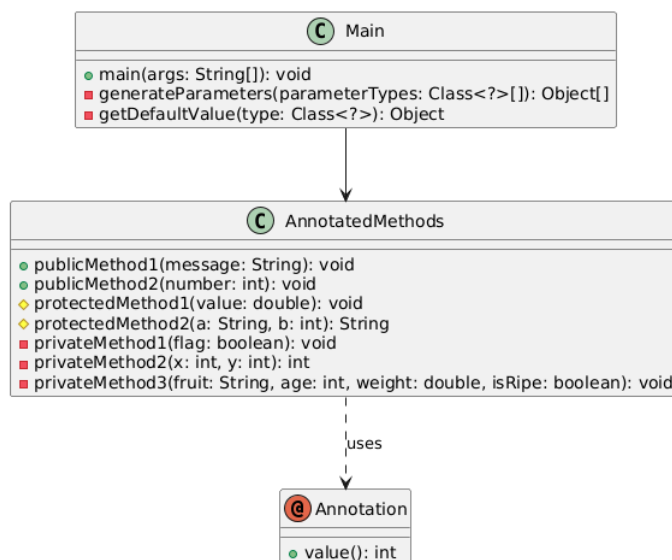
- Создан интерфейс MoveStrategy, определяющий метод move(from, to).
- Реализованы три конкретные стратегии:
 - WalkStrategy – герой идет пешком;
 - HorseRideStrategy – герой скачет на лошади;
 - FlyStrategy – герой летит.
- Класс Hero:
 - хранит текущую стратегию перемещения и текущее местоположение;
 - умеет менять стратегию в процессе игры;
 - выполняет перемещение, делегируя его выбранной стратегии.
- Класс Main:
 - запускает игру;
 - предлагает пользователю выбрать способ перемещения;
 - случайным образом выбирает следующую локацию;
 - демонстрирует работу паттерна Strategy в действии.

Так, реализована программа-игра, где герой путешествует по локациям, а пользователь может динамически менять стратегию перемещения.

ЗАДАНИЕ 2

Аннотации

Диаграмма классов



Перечень выполненных работ

- Создана собственная аннотация `@Annotation` с параметром `value` (количество вызовов метода).
- Класс `AnnotatedMethods` содержит:
 - публичные, защищенные и приватные методы;
 - часть методов помечена `@Annotation` с указанием числа повторных вызовов.
- Класс `Main`:
 - через `Reflection` получает список методов класса `AnnotatedMethods`;
 - отбирает только методы с аннотацией и с модификаторами `protected` или `private`;
 - делает их доступными (`setAccessible(true)`);
 - вызывает каждый метод указанное число раз, подставляя параметры по умолчанию;
 - выводит результаты или ошибки в консоль.

Так, реализована программа, демонстрирующая работу с пользовательскими аннотациями и `Reflection API`: методы, помеченные аннотацией, автоматически вызываются заданное количество раз.

ЗАДАНИЕ 3

Программа-переводчик

Диаграмма классов



Перечень выполненных работ

- Созданы собственные классы исключений:
 - FileReadException – для ошибок чтения файлов (файл не найден, ошибка ввода-вывода);
 - InvalidFileFormatException – для ошибок формата словаря (неверная структура строк, пустые значения).
- Реализован класс Translator, который:
 - хранит словарь (Map<String, String>), список фраз и индекс по первому слову;
 - умеет загружать словарь из файла (loadDictionary), проверяя корректность формата и выбрасывая исключения;
 - нормализует текст (нижний регистр, апострофы, дефисы, удаление лишних символов);
 - поддерживает многословные фразы, апострофы и дефисы;
 - переводит текст методом translateText, находя самые длинные совпадения из словаря и сохраняя регистр исходного текста.
- Класс Main:

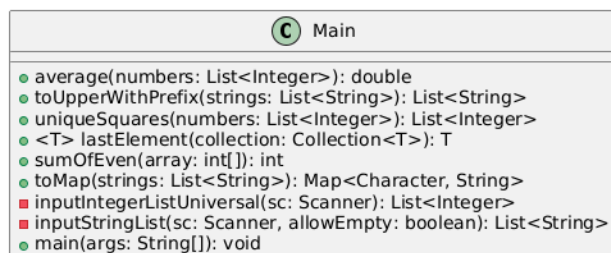
- организует консольное взаимодействие с пользователем;
- запрашивает путь к словарю, загружает его через Translator;
- предлагает два способа ввода текста: с клавиатуры или из файла;
- переводит введенный текст и выводит результат;
- корректно обрабатывает ошибки (формат словаря, чтение файлов).

Так, реализована полноценная консольная программа-переводчик, которая загружает словарь из файла, переводит текст с учетом многословных выражений и обрабатывает все возможные ошибки.

ЗАДАНИЕ 4

Stream API

Диаграмма классов



Перечень выполненных работ

- Реализован класс Main, содержащий набор методов для демонстрации возможностей Stream API:
 - average(List<Integer>) – вычисляет среднее значение списка чисел;
 - toUpperWithPrefix(List<String>) – преобразует строки в верхний регистр и добавляет префикс _new_;
 - uniqueSquares(List<Integer>) – возвращает квадраты уникальных чисел, с обработкой переполнения;
 - lastElement(Collection<T>) – возвращает последний элемент коллекции (или выбрасывает исключение, если коллекция пуста);
 - sumOfEven(int[]) – суммирует все четные числа массива;
 - toMap(List<String>) – преобразует список строк в Map, где ключ – первый символ, значение – оставшаяся часть строки (с обработкой конфликтов ключей).
- Добавлены вспомогательные методы для универсального ввода данных:
 - inputIntegerListUniversal(Scanner) – ввод списка чисел;
 - inputStringList(Scanner, boolean) – ввод списка строк (с возможностью разрешить пустой ввод).
- В методе main реализовано консольное меню, позволяющее пользователю выбрать один из методов и протестировать его работу на введенных данных.
- Обеспечена обработка ошибок:
 - ArithmeticException – при переполнении при возведении в квадрат;

– `NoSuchElementException` – при попытке взять последний элемент пустой коллекции;

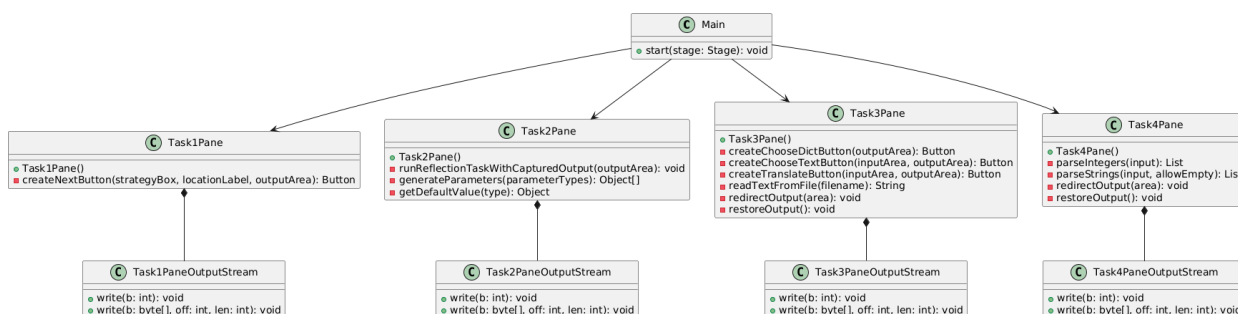
– `IllegalStateException` – при конфликте ключей в `toMap`.

Так, создана программа, демонстрирующая практическое использование `Stream API` и обработку исключений при работе с коллекциями и массивами.

КУРСОВАЯ РАБОТА

Приложение с графическим интерфейсом для заданий 1-4

Диаграмма классов



Перечень выполненных работ

В курсовой работе реализован графический интерфейс на JavaFX, который объединяет все четыре лабораторные работы в одном приложении:

- Main (GUI):
 - Наследует Application;
 - Создает главное окно с вкладками;
 - Каждая вкладка соответствует одному заданию и содержит свою панель (Task1Pane-Task4Pane);
 - Подключает CSS-стили и задает оформление.
- Task1Pane:
 - Панель для работы с lr1 (Стратегия);
 - Позволяет выбирать стратегию и управлять объектом Hero;
 - Вывод действий отображается в TextArea.
- Task2Pane:
 - Панель для работы с lr2 (Аннотации);
 - Запускает методы из AnnotatedMethods, помеченных @Annotation;
 - Результаты и ошибки выводятся в TextArea.
- Task3Pane:
 - Панель для работы с lr3 (Переводчик);
 - Позволяет выбрать словарь и файл, загрузить их и выполнить перевод;

– Использует Translator, обрабатывает исключения `InvalidFileFormatException` и `InvalidFileFormatException`.

- Task4Pane:

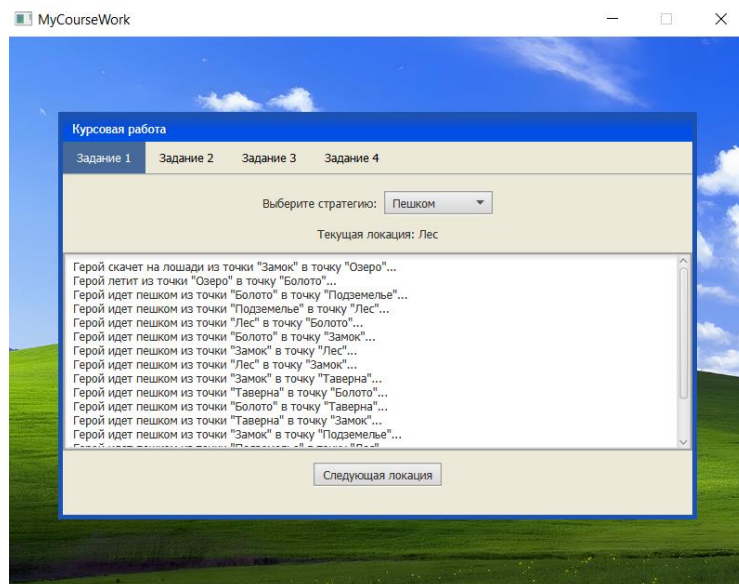
– Панель для работы с lr4 (Stream API);

– Предоставляет выбор одного из шести методов обработки данных;

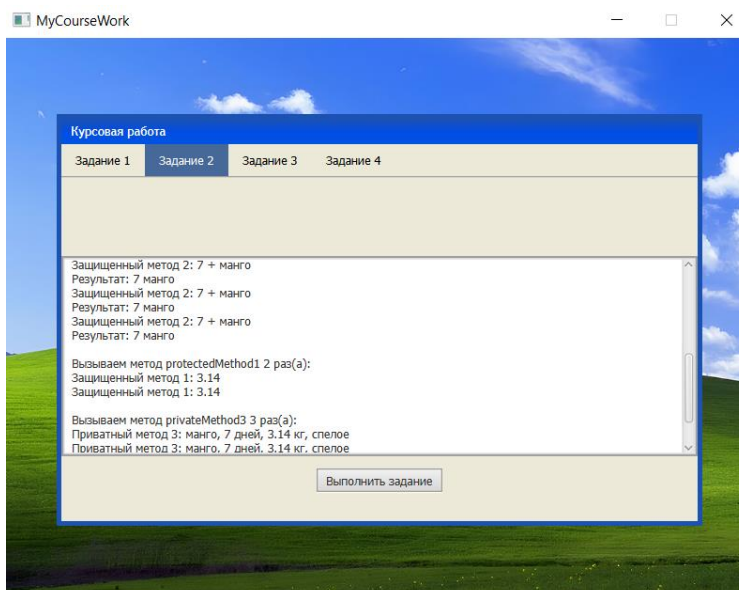
– Вызовы выполняются через статические методы класса Main из lr4.

Таким образом, GUI объединяет все лабораторные модули в единое приложение, предоставляя удобный интерфейс для их запуска и демонстрации:

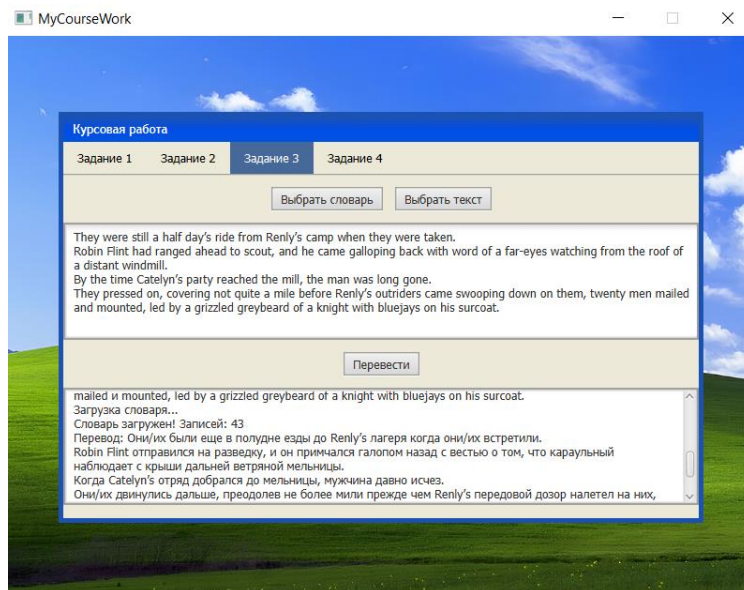
Задание 1:



Задание 2:



Задание 3:



Задание 4:

